

Тестове завдання Middle FullStack Developer

Назва тестового: Mini “Lead Tracker”, що є невеликим сервісом (mini-CRM) для ведення лідів з коментарями, пошуком, фільтрацією та пагінацією.

Час виконання: 2–3 години

Стек (обов'язково): Next.js (App Router) + NestJS + база даних **на вибір** (PostgreSQL/MongoDB) + ORM/ODM **на вибір** (Prisma/TypeORM/Mongoose тощо) + TypeScript + Docker

Ідея продукту: користувач створює “ліди”, переглядає список, фільтрує за статусом, шукає по назві/емейлу/компанії, відкриває детальну сторінку, змінює статус, додає коментарі, редагує або видаляє запис.

Сутності

1) Lead (мінімум поля)

- `name` (string, required)
- `email` (string, optional, валідний email)
- `company` (string, optional)
- `status` (enum: `NEW`, `CONTACTED`, `IN_PROGRESS`, `WON`, `LOST`)
- `value` (number, optional)
- `notes` (string, optional)
- `createdAt` (date)
- `updatedAt` (date)

2) Comment (мінімум поля)

- `leadId` (relation/reference)
- `text` (string, required, 1..500)
- `createdAt` (date)

Базу даних, тип ідентифікатора (UUID/ObjectId/auto increment) та ORM/ODM — обираєте самостійно.

Вимоги до Backend (NestJS)

1) CRUD API для Lead

`GET /api/leads` Підтримати:

- базову пагінацію: `page` , `limit`
- фільтр за статусом: `status`
- пошук: `q` (по `name` / `email` / `company`)
- сортування: `sort=createdAt|updatedAt` , `order=asc|desc`

`POST /api/leads` — створення ліда

`GET /api/leads/:id` — деталі ліда

`PATCH /api/leads/:id` — часткове оновлення (включно зі зміною `status`)

`DELETE /api/leads/:id` — видалення

2) Коментарі до ліда

- `GET /api/leads/:id/comments` — список коментарів
- `POST /api/leads/:id/comments` — додати коментар

Загальні вимоги до API

- DTO + валідація через `class-validator`
- обробка помилок:
 - `400` — некоректні дані / валідація
 - `404` — lead або comment не знайдено
- акуратна структура Nest: `Module / Controller / Service`
- Swagger/OpenAPI документація: `/api/docs`

Вимоги до Database

- реалізувати схему/моделі для `Lead` і `Comment`
- коректно зберігати `createdAt/updatedAt`

Вимоги до Frontend (Next.js, App Router)

Сторінки

`/leads`

- список лідів (таблиця або cards)
- форма створення нового ліда
- пошук `q` у UI
- фільтр за `status`

- пагінація
- відображення станів `loading / empty / error`

`/leads/[id]`

- деталі ліда
- редагування ліда (мінімум: `name`, `email`, `company`, `status`, `value`, `notes`)
- видалення ліда
- список коментарів
- форма додавання коментаря
- відображення станів `loading / empty / error`

Вимоги до інтеграції

- чиста інтеграція з API (`fetch` або `axios` — на вибір)
- базова валідація форм:
 - required поля (мінімум `name`)
 - адекватні повідомлення про помилки
- нормальна обробка API помилок (показувати текст/стан)

UI вимоги

- простий, але охайний інтерфейс (Tailwind)
- без “порожніх” екранів
- обов’язково показати стани: `loading / empty / error`

Не робити

- авторизацію, ролі, ACL
- складні файлові завантаження
- надмірну/важку архітектуру
- сторонні сервіси (платежі, інтеграції, аналітика)
- “todo-list” по суті (чекбоксы/таски). Тут саме міні-CRM для лідів.

Acceptance критерії (що перевіряємо)

- коректність CRUD
- валідні статус-коди та помилки (400/404)
- охайний TypeScript (типи, неймінг, структура)
- зрозуміла структура Nest (DI, модулі, відповідальності)
- читабельні компоненти Next (без хаосу)
- робочий пошук/фільтр/пагінація
- Docker піднімає проєкт без “танців”
- адекватні коміти та базова якість коду

Формат здачі

- GitHub репозиторій: або **монорепо** (на ваш вибір)
- обов'язково: [README.md](#)
- обов'язково: **Dockerfile**(и) + `docker-compose.yml`

Що має бути в [README.md](#) (обов'язково)

1. Як запустити локально:

- команди для backend і frontend
- команди для запуску через Docker (`docker compose up --build`)

2. Змінні оточення:

- `backend/.env.example`
- `frontend/.env.example`
- коротко описати ключі (DB URL, API URL тощо)

3. Як перевірити API:

- короткі приклади ендпоінтів (мінімум список Lead + створення Lead + додати коментар)
- посилання на Swagger `/api/docs`

4. Як зробити build та запустити в prod-режимі:

- команди `build` + `start` для backend і frontend (або через Docker)

Додатково (бажано)

- посилання на тестовий деплой:
у README додайте:

- - Demo URL (web)
 - Demo URL (api)
 - Swagger URL

Комунікація

Якщо щось не встигаєте — зафіксуйте в README блок:

“Що не встиг(ла) і як би доробив(ла)” (2–5 рядків).